

Homework 8: NP-Completeness

Will Griffin

April 26, 2026

1. Neural Network

Prove that it is NP-complete to decide, given a neural network with l inputs and m hidden units, whether there is any sequence of inputs $x_1, \dots, x_l$ that makes $y = 1$.

To prove this is NP-complete, first we need to show this problem is in NP. Construct a verifier V such that $V =$ “On input $\langle X, U, V, A, b \rangle$,

1. Compute each h_j for $j = 1, \dots, m$
2. Compute y using each h_j ”

where

- X is the set of inputs $X = \{x_1, \dots, x_l\}$
- U is the set of weights $U = \{u_{ij} \mid i \in \{1, \dots, l\}, j \in \{1, \dots, m\}\}$
- V is the set of weights $V = \{v_1, \dots, v_m\}$
- A is the set of weights $A = \{a_1, \dots, a_m\}$
- b is a weight

Since step 1 runs in $O(ml)$ time and step 2 runs in $O(m)$ time, this verifier runs in $O(ml)$ time. Therefore, this problem is in NP.

Next, show that 3-SAT reduces to this neural network problem.

Create a mapping f where f takes a boolean formula ϕ in 3-CNF with n variables $z_1, \dots, z_n$ and K clauses $C_1, \dots, C_K$ and constructs a neural network with l inputs and m hidden units.

Let $l = n$ and $m = K$.

Claim: ϕ has some certificate of variables $z_1, \dots, z_n$ that makes ϕ true iff the neural network has some set of inputs $x_1, \dots, x_l$ that makes $y = 1$.

For any set of variables $z_1, \dots, z_n$ that makes ϕ true, we construct a neural network with weights u_{ij} such that if z_i appears in C_j , $u_{ij} = 1$ and if $\neg z_i$ appears in C_j , $u_{ij} = -1$. Otherwise, $u_{ij} = 0$. Let the number of negated variables in C_j be equal to N_j . Set $a_j = N_j - 0.5$. Therefore, for a given j , the sum becomes the count of satisfied literals -0.5 . Therefore, if no literals are satisfied (the clause overall is false), $h_j = 0$. Otherwise, if at least one literal is satisfied (the clause overall is true), $h_j = 1$. Then, since we want $y = 1$ if all the clauses are true, set $v_j = 1$ for all $j = 1, \dots, m$ and $b = 0.5 - m$. Therefore, $y =$ the count of all true clauses minus the count of all clauses $+0.5$. If all clauses are true, $y = h(0.5) = 1$.

For any set of inputs $x_1, \dots, x_l$ that make the neural network $y = 1$, we know from the formula that the input had to be positive. Therefore,

$$\sum_{j=1}^m h_j v_j + b > 0 \implies \sum_{j=1}^m h_j - (m - 0.5) > 0 \implies \sum_{j=1}^m h_j > m - 0.5$$

Since each h_j is either 0 or 1, for their sum to be greater than $m - 0.5$, each $h_j = 1$ for all $j = 1, \dots, m$. Since all $h_j = 1$, this means each clause has at least one satisfied literal. Since each clause has one satisfied literal making it true, this means ϕ is true.

Clearly, f runs in polynomial time.

Since the neural network problem is in NP, and 3-SAT can be reduced to the neural network problem, it is NP-complete.

2. Digits

Prove that $\{+, \times\}$ -DIGITS is NP-complete:

$$\{+, \times\}\text{-DIGITS} = \{\langle t, S \rangle \mid (t, S, \{+, \times\}) \text{ is solvable}\}$$

Assume that the size of $\langle t, S \rangle$ is the total number of bits in t and S .

First, to prove this NP-complete, we first prove $\{+, \times\}$ -DIGITS is in NP by creating a verifier V such that $V =$ "On input $\langle t, S, C \rangle$, where $C \subseteq S$,

1. Start a total at 0. For every integer a in C , either add a to the total or multiply the total by a .
2. If the total equals t , *accept*. Otherwise, *reject*.

Clearly V runs in $O(N)$, where N is the size of $\langle t, S \rangle$. Therefore, $\{+, \times\}$ -DIGITS is in NP.

Next, show that SUBSET-SUM reduces to $\{+, \times\}$ -DIGITS.

Create a mapping f where f takes a set of integers S and a target value t and creates a new set of integers S' and a new target value t' where

$$\begin{aligned} S' &= \{b^i + a_i \mid \text{for each } a_i \in S\} \cup \{b^i \mid \text{for each } a_i \in S\} \\ t' &= \left(\sum_{i=1}^j b^i \right) + t, j = |S| \\ b &= \left(\sum_{i=1}^j a_i \right) + 1, j = |S| \end{aligned}$$

Claim: S has a subset of integers that sum to t iff S' has a subset of integers that sum and multiply to t' .

Given a set of integers S and a target sum t , assume there is a set of indices I such that $\sum_{i \in I} a_i = t$. The expression and value can be constructed as such:

$$\begin{aligned} \text{Expression} &= \sum_{i \in I} (b^i + a_i) + \sum_{i \notin I} b^i \\ \text{Value} &= \sum_{i=1}^j b^i + \sum_{i \in I} a_i \end{aligned}$$

However, since $\sum_{i \in I} a_i = t$, the value becomes $\sum_{i=1}^j b_i + t = t'$. Therefore, we can construct some S' that has a subset that sums to t' .

Given a set of integers S' and target sum-product t' , recall $t' = \sum_{i=1}^j b_i + t$. If we multiply any two numbers from S' , we get a term involving b^{k+l} . If $k+l > j$, a multiplication result would be greater than t' . If $k+l \leq j$, the multiplication would create "cross-terms" (i.e. $a_k \times b^l$) that cannot be offset by addition since $b > t$. Therefore, any solution to t only uses addition.

Therefore, our expression becomes

$$\sum (\text{selected } b^i + a_i \text{ and } b_i) = \sum_{i=1}^j b^i + t$$

Let I be the subset of indices i where we choose to add $b^i + a_i$. This means that $\sum_{i \in I} a_i = t$. Therefore, from a set S' and sum-product t' , we can construct a set S and a sum target t .

Clearly f runs in polynomial time.

Since $\{+, \times\}$ -DIGITS is in NP and SUBSET-SUM reduces to $\{+, \times\}$ -DIGITS, $\{+, \times\}$ -DIGITS is NP-complete.

3. Trapdoors and Lava

Prove that it is NP-complete to decide, given a room as defined in HW8, whether there is a subset of buttons that you can push to make a safe path from the entrance to the exit.

To show that the trapdoor problem is NP-complete, first we show that it is in NP.

Construct a verifier V such that $V = \text{"On input } \langle R, s, t, \Sigma \rangle,$

1. For each symbol a in Σ , press the corresponding a button.
2. Perform a traversal of the room. If the exit t is reachable from the entrance s , *accept*. Otherwise, *reject*.

Clearly V runs in polynomial time.

Next, show that 3-SAT reduces to the trapdoor problem.

Create a mapping f such that f takes some boolean formula ϕ in 3-CNF with n input variables $x_1, \dots, x_n$, and K clauses $C_1, \dots, C_K$, and generates a room R with entrance s , exit t , and button alphabet Σ . For each variable x_i , add a button b_i to Σ such that pushing b_i is equivalent to x_i being true and not pushing b_i is equivalent to x_i being false. For each clause C_j , define a 3×3 subgrid such that passage from beginning to end is ensured if at least one literal is satisfied (i.e. a column of 3 unlabeled white trapdoors, a column of 3 labeled trapdoors white/red according to their value, and a column of 3 unlabeled white trapdoors). For each literal $l = x_i$ in C_j , create a red trapdoor with label b_i . For each literal $l = \neg x_i$ in C_j , create a white trapdoor with label b_i . Finally, for the overall room R , construct a grid of connected gadgets, where the exit of gadget G_j is connected to the entrance of gadget G_{j+1} using static white trapdoors with no symbol and fix the entrance s to the start of gadget G_1 and the exit to the end of gadget G_K .

Claim: ϕ has some set of variables $x_1, \dots, x_n$ that makes ϕ true iff the room R has some set of symbols C that when pressed, allow the room to be traversed from entrance to exit.

For a set of variables $x_1, \dots, x_n$ that satisfies ϕ , for every variable x_i , if x_i is true, push button b_i . If x_i is false, do nothing. Therefore, for a positive literal x_i , pushing b_i turns a red trapdoor into a

white trapdoor. For a negative literal $\neg x_i$, not pushing b_i keeps the white trapdoor. Since T satisfies ϕ , this means that every clause C_j contains at least one literal that evaluates the clause to true. Therefore, in each gadget, there exists at least one connecting white trapdoor to reach the internal gadget exit from the internal gadget entrance.

For a room R with buttons to press C , all buttons Σ entrance s , and exit t that is traversable, we know that each gadget is able to be passed through since the overall traversal is possible. Therefore, at least one literal in each gadget clause is true. We derive the truth assignment for variables such that for each $b_i \in \Sigma$, if $b_i \in C$, x_i is true and if $b_i \notin C$, x_i is false. Therefore, since we know that each gadget literal is able to be passed through, and we get variable values $x_1, \dots, x_n$ from buttons $b_1, \dots, b_n$, ϕ is satisfiable.

Clearly f runs in polynomial time.

Since the trapdoor problem is in NP and 3-SAT reduces to it, the trapdoor problem is NP-complete.